PureScript backends: a comparison

Seed material for a future comparison site covering the whole backend family. This table covers the five backends we have local checkouts and working knowledge of: the reference JS backend in its two habitats (browser, Node), purerl (Erlang/BEAM), purescript-julia (Jurist), and katsujukou's purescript-backend-wasm (Wasm GC). To be added as the site project spins up: purescript-native (Go, cloned at `purescript-backends/purescript-native`), the from-scratch Python backend (`purescript-python-new`), purs-backend-es (the optimizing JS backend — same habitat as JS, different generator, which makes it a useful control column), and Fabrizio's upcoming Racket backend.

A note on framing: "backend" mixes two axes — *code generator* and *runtime habitat*. Browser and Node share purs's JS code generator and differ only in runtime and FFI ecosystem; purerl and Jurist are separate code generators consuming CoreFn. The table keeps all four columns because the *experienced* differences (FFI surface, concurrency, what programs make sense) follow the habitat, not just the generator.

The table

	JS (browser)	JS (Node)	purerl (BEAM)	Jurist (Julia)	Wasm (katsujukou)
Code generator	purs itself (CoreFn → CoreImp → optimized ES modules)	same	standalone `purerl`, consumes CoreFn JSON	standalone `purejl` (Haskell), consumes CoreFn JSON	standalone compiler **written in PureS** consumes CoreFn JSON + externs.cbo Wasm GC module via Binaryen
Status	reference, definitional	reference	production-mature (id3as media streaming); maintained package sets	experimental, weekend-shaped; core libs working	experimental, advancing fast (25 ADRs benchmarks); agent-assisted developr
Semantics vs reference	— is the reference	—	diverges by design (Int, strings); own test suites	**422/426 differential tests byte-identical**; 4 documented divergences	same-source benchmarks vs purs JS a backend-es; Int is i32 (JS-aligned)
Functions	curried unary; optimizer inlines/uncurries hot paths	same	curried funs; arity-optimized top-level variants	curried unary closures, `(f)(x)(y)`	uniform `eqref` calling convention; part application supported; aggressive inlin order specialization
ADT values	constructor functions + `instanceof` dispatch, fields `value0…`	same	tagged tuples, atom tag: `{just, X}`	tag-tuples: `("Just", x)`, tag at `[1]`	struct *subtypes* of a tag-only `$Data` b like ADTs as unboxed `i31`; scalar fields struct
Newtypes	erased	erased	erased	erased	erased
Records	JS objects	same	Erlang maps (atom keys)	`Dict{String,Any}` + `merge`	`$Rec` struct of parallel arrays: interned values; polymorphic update (ADR-002
Typeclass dictionaries	JS objects; common instances inlined by the optimizer	same	maps	`Dict{String,Any}` keyed by member name	**eliminated** — positional specialization recursive instance groups handled
Int	double wrapped `|0` → int32	same	**bignum** (arbitrary precision)	**Int64** (Bits/pow do apply JS `ToInt32`)	**i32** (true 32-bit, JS-aligned); boxed `$I` unboxed to raw `i32` by the optimizer
Number	IEEE double	same	float (double)	Float64; `show` reproduces JS `toString` placement rules	f64 (boxed `$Num`, unboxed by the opti
Strings	UTF-16 code units	same	UTF-8 binaries	UTF-8 `String`; CodeUnits API is codepoint-based (BMP-identical)	UTF-8 bytes in a Wasm GC array
Effect	nullary thunk; MagicDo collapses binds	same	nullary funs	nullary thunk	native lowering — collapses to constar loops; whole-program purity analysis (0015/0018/0019)

	JS (browser)	JS (Node)	purerl (BEAM)	Jurist (Julia)	Wasm (katsujukou)
TCO	purs optimizer: self-tail-calls → `while` loops	same	**native BEAM TCO, including mutual recursion**	trampoline mirroring purs's optimizer: self-tail-calls → dispatch loops (verified 10^8)	tail-call elimination in codegen; survive recursion where both JS backends ove (`bintreeBfs`)
Mutual recursion (unbounded)	MonadRec idiom	same	free (native TCO)	MonadRec idiom (matches JS)	TCE in codegen (extent: see ADRs)
Lazy/recursive bindings	runtime lazy thunks	same	similar runtime support	`_runtime_lazy` thunks, smart Rec partition	recursive let-bindings supported; pure globals instantiated at start (ADR-000
FFI unit	`.js` ES module per PS module	same	`.erl` module per PS module	`Module_foreign.jl` included *inside* the generated module	curated `ulib/` `.wat` per core module with marshalling from **reconstructed f signatures** (externs.cbor, ADR-0016)
Concurrency story	event loop, workers	event loop, worker_threads	**processes + OTP supervision — the raison d'être**	Tasks/threads available; designed role is a *leaf service*, not coordinator	the host's (browser/Node); fully sandb
Native niche	DOM/UI (Halogen, react bindings)	servers, CLIs, tooling	soft-realtime distributed systems, live supervision trees	numerics: hot kernels in Julia FFI leaves (DiffEq, DynamicalSystems, Catlab), PS as typed thin skin	portable sandboxed compute; hot kerr *same habitat as JS* — the performance
Library coverage	entire registry	entire registry	large curated package sets	prelude/effect/console/arrays/st/strings/foldable-traversable/integers/numbers/unfoldable/enums; Regex stubbed	curated `ulib` subset (Array/Eq/Ord/Show/Foldable/Functor/I ...)
Perf shape	V8 JIT is excellent for closure-heavy code	same	not numeric; superb latency/IO	curried-Dict glue ~325 ns/iter (measured); numerics belong behind the FFI seam	fastest of three on every benchmark; 5 allocation/pattern-match-heavy, ~1.6× (steady-state, post-warmup)
In this ecosystem	Hylograph showcases, Halogen apps	HTTPurple APIs, build tooling	purerl-tidal: TidalCycles scheduling on BEAM	Hylograph compute leaves (Marginalia 219)	newly cloned (`purescript-backends/purescript-backend-wa` candidate matrix column; benchmark r to adopt

Reading the family

The interesting pattern: each non-JS backend exists to borrow a *runtime virtue* the JS engines don't have, while keeping PureScript's type system as the lingua franca.

- **purerl** borrows BEAM's process model — supervision, distribution, soft-realtime scheduling. Its semantic divergences (bignum Int, binary strings) are *upgrades along the BEAM grain*, accepted rather than papered over.
- **Jurist** borrows Julia's numeric stack — the JIT, the array ecosystem, DiffEq/DynamicalSystems/Catlab. Its divergences (Int64, codepoint strings) follow the same philosophy: take the host's better number and string types, document the seam, and prove everything else identical with a differential suite.
- **purescript-backend-wasm** is the novel case: it borrows a *substrate* virtue (Wasm GC structs, sandboxing, portability) and competes **in the same habitat as JS** — beating V8's JS output on V8 itself (5–8× on allocation-heavy benchmarks) via representation: unboxed scalars, struct-subtyped ADTs, eliminated dictionaries. The first family member whose differentiation is *performance*, not habitat.
- The JS backend's virtue is *being everywhere* — and being the semantic reference the others measure against.

The same lens will apply to the additions: Go (static binaries, goroutines), Python (ubiquity, data tooling), Racket (macros and language-building).

For the comparison site (separate project)

- Columns to add: purescript-native (Go), purescript-python-new, purescript-racket (when it lands).
- Each cell wants a footnote link to evidence — for Jurist, most rows link to `test-suite/` results or README sections.
- A "run the same program" strip — one small PS module, its output and timing on every backend — would make the table falsifiable, in the spirit of the differential suite.

Adding a backend column to the differential suite

Instructions for an agent session tasked with extending the cross-backend differential matrix (see `backend-comparison.md` and Marginalia projects 134/220) to purerl, purepy, or purescript-go. Written after building the Julia column; the lessons below are the ones that cost time.

Where the work happens

This file lives in the **site repo** (`purescript-polyglot`, the future polyglot.purescri.pt) because the matrix and its results are family-level content. But each backend column is built by an agent session running **in that backend's own home context** — full local knowledge of the language, its toolchain, and its quirks:

Column	Work from	Why there
purerl	`purescript-backends/purerl` (and consult `music/live-coding/purerl-tidal` for a working spago/package-set/FFI setup)	the live purerl deployment in this ecosystem
purepy	`purescript-backends/purescript-python-new`	its own cross-backend harness is the prior art
Julia	`purescript-backends/purescript-julia` (Jurist)	reference implementation of this whole process
psgo	`purescript-backends/purescript-native` (cloned 2026-06-07)	the Go column
wasm	`purescript-backends/purescript-backend-wasm` (cloned 2026-06-07)	katsujukou's Wasm GC backend — see its dossier below

Results — runners, divergence entries, comparison-table columns, session notes — land back **here**. The harness itself currently lives in Jurist's `test-suite/` (it predates this repo's involvement); part of the site build is migrating/generalizing it into this repo per the structure sketch on Marginalia 220 (`shared/ programs/ divergences/ native/ harness/ site/`). Until then, treat Jurist's `test-suite/run_tests.py` as the canonical harness and extend it in place. Jurist's own copy of the suite stays regardless — it is that backend's verification artifact.

Prior art shelf: `docs/kb/research/purescript-alternative-backends-comparison.pdf` in this repo predates this effort — check it for dimensions the table is missing.

The contract

"Adding a backend" means, concretely:

1. **Build path**: compile `test-suite/src/Test/*.purs` to runnable artifacts on your backend. The suite is deliberately FFI-free beyond the core libraries, so the only foreign code you need is the backend's own core-library shims.

2. **Runner**: a `run_<backend>(module) -> (stdout, error)` function in `test-suite/run_tests.py`, plus the module-name mapping (see table below). Runners must be pure subprocess calls — no shared state.
3. **Divergence curation**: every non-identical line either becomes a fix (your shim is wrong) or a `KNOWN_DIVERGENCES` entry with a prefix naming the cause (`INT64-`, `ASTRAL-`, `BIGNUM-`, `FLOATFMT-`, …) AND a sentence in the backend's README. An uncurated divergence is a failure. **Counterexamples are first-class content** — the site's thesis is that divergences are enumerable, so finding one is a result, not an embarrassment.
4. **Results as data**: emit one JSONL record per (program, backend, test) — the site renders from this. Don't print prose summaries only.
5. **Regression**: the existing columns (Node, Julia) must still pass.

Module-name mappings:

PS module	JS	Julia	Erlang (purerl)	Python (purepy)
`Test.Arrays`	`output/Test.Arrays/index.js`	`Test_Arrays`	`test_arrays@ps`	snake-cased module in `output-py/`

`main` is an `Effect Unit` — a nullary closure on every backend. You must *apply* it: `m.main()` (JS), `Test_Arrays.main()` (Julia — the generated value is the thunk), `('test_arrays@ps':main())()` (Erlang — note the double application: fetch, then run).

Ground rules (learned the hard way)

- **Read the real JS foreign modules** (`.spago/p/<pkg>/src/**/*.js`), never another backend's shims. The python backend's shim names were cargo-culted at least once. The JS file is the spec; the `.purs` `foreign import` declarations give you arities (`Fn4` = genuine 4-arg function if your representation uncurries them).
- **JS arity tolerance does not port.** JS foreigns call `f()` on functions that CoreFn types as 1-ary (the Partial dict, `mkFn0`'s `Unit -> a`). Your shim must pass the argument explicitly (`f(nothing)` in Julia). Grep the JS for zero-arg calls.
- **Some "foreign-looking" names are PS-defined.** `mkFn1`/`runFn1` are written in PureScript; shimming them collides with the generated module body. Trust the corefn.json `foreign` array, nothing else.
- **The reference is not always "right".** `sumTo 0 1000000` *overflows* on the JS backend (int32 wrap). When your backend's answer is better, document the divergence with a prefix — don't cripple your runtime to match, and don't silently diverge. BEAM bignum and Python int will hit this immediately: expect the `INT64-` tests to need a third expected value (the suite currently encodes JS-vs-Julia; turn the entry into per-backend expectations when you add a third).
- **Number formatting is the biggest hidden surface.** JS `Number.prototype.toString` placement rules (decimal within 1e-6 ≤ |n| < 1e21, exponential outside, `.0` suffix via Show) had to be reimplemented for Julia (`_js_number_string` in the runtime). Erlang's and Python's default

float printing differ from JS in exactly these zones. Run `Test.Numbers` early — it will tell you whether your backend's `showNumberImpl` needs the same treatment.
- **Sort stability is tested** (`sort-stable`). JS `sortByImpl` is a stable merge sort. Check what your backend's shim actually does.
- **`ordArrayImpl`'s length tiebreak is inverted** (longer → -1; the PS caller re-inverts via `compare 0 _`). We shipped this bug; the suite caught it. Check your backend's prelude shims for the same trap — any foreign whose sign convention is consumed inverted.
- **stack snapshot reuse**: if you build a Haskell-based backend (purerl, purepy), copy `stack.yaml` AND `stack.yaml.lock` from a sibling that already builds — the lock pins the GitHub tarball hash that determines the snapshot key. Without it: ~25 min rebuild; with it: ~90 s.

purerl (Erlang/BEAM)

- **Toolchain**: local source checkout at `purescript-backends/purerl` (stack project, exe `purerl`); a prebuilt npm binary also exists at `music/live-coding/purerl-tidal/node_modules/.bin/purs-backend-erl`. Erlang/OTP is installed (`/opt/homebrew/bin/erl`, `erlc`).
- **CRITICAL — package sets**: purerl does NOT compile against the registry's JS core libraries. It needs the purerl forks, via its own package set. See `music/live-coding/purerl-tidal/spago.yaml` for a working example: `workspace.packageSet.url: https://raw.githubusercontent.com/purerl/package-sets/erl-0.15.3-20220629/packages.json` and `backend.cmd: purs-backend-erl` (or `purerl`). Consequence: the suite needs a **second workspace** (`test-suite-erl/`) with its own `spago.yaml`; share the PS sources by relative glob or symlink to `../test-suite/src`. Check whether the pinned set's compiler version matches the installed purs (0.15.x) before debugging anything else.
- **Build & run**: `spago build` emits `output/<Module>/<module>@ps.erl` (purerl is invoked per-module on corefn). Then: `erlc -o ebin output/*/*.erl` and `erl -pa ebin -noshell -eval "('test_arrays@ps':main())(), init:stop()."`
- **Expected divergences**: Int is bignum (`INT64-` tests give exact answers like Julia — note JS is the odd one out, the entry needs per-backend expectations); float printing (Erlang's `io_lib:format`-based Show may differ in exponent zones — discover differentially); possibly string Show escaping. Strings are UTF-8 binaries; the ASTRAL- tests should agree with Julia, not JS.
- The purerl core libs are mature — expect FEW shim bugs; most divergence will be deliberate grain.

purepy (purescript-python-new)

- **Toolchain**: sibling repo `purescript-backends/purescript-python-new` (stack project, exe `purepy`). Its own `test-project/` contains `cross_backend_test.py` — **prior art for this exact task**, with a curated KNOWN_DIVERGENCES set (emoji/code-unit entries). Port those entries; don't rediscover them.
- **Build & run**: spago with `backend.cmd: "true"` for corefn (same trick as Jurist), then `purepy output output-py`. Modules are snake-cased files; run via `python3 -c "import sys; sys.path.insert(0,'output-py'); import test_arrays; test_arrays.main()"`. Verify the exact snake-casing purepy produces for `Test.ADTs` (historically `test_cross_backend_a_d_ts` — acronyms split oddly).

- **Audit while you're there**: purepy's shims predate the read-the-real-JS rule. Validate its foreign names against corefn.json `foreign` arrays for every module the suite touches. Fixing purepy is in scope and valuable.
- **Expected divergences**: Python int is bignum (same as BEAM); float repr differs from JS (e.g. 1e16 zone, `inf` vs `Infinity` if unshimmed); `ASTRAL-` tests agree with Julia.

purescript-go (purescript-native)

- **Local clone**: `purescript-backends/purescript-native` (driver: `psgo`). Also clone its companion Go FFI when starting: `github.com/purescript-native/go-ffi`. Needs a Go toolchain (`brew install go` if `which go` fails).
- psgo consumes corefn and produces **one executable per Main** — ten test modules would mean ten builds. Preferred integration: add a dispatcher `Main.purs` to the suite that switches on its first CLI arg (`main = getArgs >>= case _ of ["Test.Arrays"] -> Arrays.main; …` — needs an argv foreign or `node-process`-equivalent per backend, so weigh it; compiling N small mains may honestly be simpler).
- Expect the *least* coverage here: go-ffi tracks an older core-library era. Budget time for missing shims, and treat each as a finding for the comparison table's "library coverage" row.
- **Expected divergences**: Int is int (64-bit on arm64) — Julia-like; float formatting via strconv differs from JS; strings UTF-8.

wasm (katsujukou/purescript-backend-wasm)

- **Local clone**: `purescript-backends/purescript-backend-wasm`. Toolchain: pnpm + Node (the compiler is **written in PureScript**, drives the Binaryen JS API; pinned purs v0.15.16 — note our other work uses 0.15.x too, but check the pin). Start with the repo's own `CLAUDE.md`, `docs/compilation-pipeline.md`, and `docs/interop.md` — this is the best-documented backend in the family (25 ADRs).
- **Inputs**: CoreFn JSON **and externs.cbor** (it reconstructs foreign signatures from externs for boundary marshalling, ADR-0016). Output: a single linked Wasm GC module; run under Node (V8) with the host shim — see `bin/` and `examples/` for the actual invocation.
- **Suite feasibility — set expectations**: its `ulib/` curated FFI covers a *subset* of the core libraries (Array/Eq/Ord/Show/Foldable/ Functor/Int/CodeUnits, ...). The differential suite's ten modules will exceed that coverage initially. Run module-by-module, record SKIPPED honestly, and treat each gap as a "library coverage" data point — the matrix is allowed to have holes; silent shrinkage is the only failure.
- **Expected divergences**: few in principle — Int is true i32 (JS-aligned, so `INT64-` tests should match *JS*, not Julia); strings are UTF-8 internally but the CodeUnits API semantics need differential discovery; Number formatting at the JS boundary (host-side `toString` vs in-module formatting) is worth probing early, as ever.
- **What to learn while there** (applies to the whole family):
  - **ADR discipline** — every representation/optimization decision recorded with status and date. Adopt for Jurist.
  - **externs.cbor signature reconstruction** — typed FFI marshalling. Directly relevant to Jurist's Tier-1 typed arrays (knowing a foreign is `Vector Number` permits `Vector{Float64}`).
  - **Benchmark methodology** — the same source timed across three code generators, steady-state post-warmup, graphs in-repo (`bench/`). This *is* the site's "run the same program

everywhere" strip, already practiced; adopt its shape.
    - purs-backend-es appears as its comparison baseline — that optimizing JS generator belongs in our matrix as a control column (same habitat, different generator).

Harness integration

Refactor `run_tests.py` from hardcoded `run_js`/`run_julia` to a registry:

```
BACKENDS = {
    "js":     Backend(build=build_js, run=run_js, ref=True),
    "julia":  Backend(build=build_julia, run=run_julia),
    "erl":    Backend(...),
    ...
}
# --backends erl,julia ; default: all available (probe toolchains, skip
# missing with a SKIPPED record, never a silent drop)
```

Diff every backend against the reference, AND pairwise where both diverge from JS the same way (bignum backends should agree with each other — that's a checkable claim). KNOWN_DIVERGENCES becomes `(module, test) -> {backend_or_class: expected}` — divergence classes (`bignum`, `utf8`) beat per-backend entries where they apply.

Definition of done

- All ten `Test.*` modules run on the new backend; counts reported.
- Zero uncurated divergences; every curated one has a prefix, a per-backend (or per-class) expectation, and a README sentence.
- JSONL results emitted; existing columns still green.
- `backend-comparison.md` column updated from observed facts (not documentation folklore) — cite test names in cell footnotes where the table makes a checkable claim.
- Worklog + Marginalia note (project 220) per ecosystem conventions.